

오프닝

안녕하세요. 오늘은 기술 리서치 프로젝트로 진행한 **ETF Atlas** 를 소개하면서, 그 과정에서 사용한 기술과 부딪힌 문제들을 이야기하려 합니다.

문제 제기

한국 증시에 상장된 ETF는 약 1,175개입니다. 그중 국내 주식형이면서 순자산 500억 원 이상인 것만 골라도 190개, 순자산 합계 약 156조 원이고, 이 ETF들이 담은 종목은 수천 건의 보유 관계를 이룹니다. 여기서 의미 있는 정보를 뽑으려면 **관계를 탐색** 해야 합니다.

"삼성전자를 가장 많이 보유한 ETF는?"이라는 질문은 관계형 DB라면 **JOIN** 여러 개에 **GROUP BY**, **ORDER BY** 가 필요합니다. "KODEX 200과 비슷한 ETF는?"은 자기 조인과 집계가 겹칩니다. **쿼리가 복잡해질수록 성능과 가독성이 함께 떨어 집니다.**

여기에 욕심을 하나 더 냈습니다. 이런 질문을 개발자가 아닌 사람도 할 수 있으면 좋겠다는 것입니다. "삼성전자와 SK하이닉스가 많이 든 ETF를 수수료순으로 보여줘"라고 **자연어로 물으면 알아서 데이터를 찾아 답하는 챗봇** 을 만들고 싶었습니다.

복잡한 관계 탐색, 자연어 질의, 그리고 매일 쌓이는 시장 데이터의 자동 수집. 이 세 문제를 풀려고 고른 기술이 **Apache AGE, LLM 에이전트 프레임워크, Apache Airflow**입니다.

기술 선택 배경

주제는 평소 관심 있던 ETF 투자로 잡았습니다. 실제 서비스에 가까운 도메인에서 써 봐야 리서치 결과가 의미 있다고 생각했기 때문입니다.

- **그래프 데이터베이스** — ETF 데이터는 본질적으로 관계 중심입니다. 별도 그래프 DB 대신 PostgreSQL 확장인 **Apache AGE** 를 골랐습니다
- **LLM 에이전트** — 처음에는 경량 프레임워크 **smolagents** 의 CodeAgent로 시작했고, 지금은 **pydantic-ai** 기반 tool-calling 에이전트로 바꿨습니다. 이 전환이 오늘 이야기의 한 축입니다
- **데이터 파이프라인** — 배치 중심 워크로드라 Python 네이티브인 **Apache Airflow** (현재 3.x)를 선택했습니다

데이터 소스는 **한국투자증권 KIS Open API** 로 통일했습니다. 초기에는 KRX Open API와 pykrx를 함께 썼지만 지금은 모두 걷어냈습니다.

이 기술들이 실제 서비스 수준의 프로젝트에서 얼마나 쓸 만한지, 그리고 쓰다 보니 무엇을 바꾸게 됐는지를 순서대로 공유하겠습니다.

Apache AGE 기술 소개

Apache AGE란?

Apache AGE 는 PostgreSQL에 그래프 데이터베이스 기능을 더하는 확장(Extension)입니다. 이름이 "A Graph Extension" 의 약자입니다. 이 프로젝트는 PostgreSQL 18 위에 AGE 1.8.0을 올려 씁니다.

- PostgreSQL 위에서 동작 — 별도 DB 서버 없이 확장만 설치하면 됩니다
- Cypher 쿼리 지원 — Neo4j에서 쓰던 Cypher 문법을 대부분 그대로 씁니다
- SQL과 Cypher 혼용 — 관계형 테이블, 그래프, 그리고 pgvector 임베딩까지 한 DB에 둘 수 있습니다

라이선스는 Apache 2.0 입니다.

Neo4j vs Apache AGE

Neo4j

- 전용 그래프 DB, 성능과 기능 면에서 가장 성숙
- Community / Enterprise 에디션으로 나뉘고 Enterprise는 상용 라이선스
- 별도 서버 운영 필요
- RDBMS와 데이터를 따로 관리

Apache AGE

- PostgreSQL 확장 — 기존 인프라 그대로
- Apache 2.0 라이선스
- 한 DB에서 관계형 + 그래프 + 벡터
- Cypher 지원이 불완전하고 에코시스템이 작음

이미 PostgreSQL을 쓰고 있고 그래프 쿼리가 워크로드의 일부라면 AGE가 합리적인 선택입니다.

AGE 동작 방식

일반 SQL 안에서 `cypher()` 함수를 호출하는 형태입니다.

```
SELECT * FROM cypher('etf_graph', $$
  MATCH (e:ETF {code: '069500'})-[h:CURRENT_HOLDS]-(s:Stock)
  RETURN {stock: s.name, weight: h.weight}
  ORDER BY h.weight DESC LIMIT 10
$$) as (result agtype);
```

주의할 점: 그래프 이름을 지정해야 하고, 결과 타입은 `agtype` 입니다. 반환 컬럼을 하나로 선언했으니 여러 값을 돌려줄 때는 맵으로 감싸야 합니다.

써 보며 만난 AGE의 특성

- 같은 WITH 절에서 만든 별칭으로는 ORDER BY를 못 합니다 — 집계 결과로 정렬하려면 WITH 를 한 번 더 뒤야 합니다
- 쿼리가 SQL의 `$$ ... $$` 안에 들어갑니다 — 쿼리에 `$$` 가 섞이면 따옴표가 닫히고 그 뒤가 날 SQL로 해석됩니다. LLM이 쓴 쿼리를 실행할 때 특히 중요합니다(AI 챗봇 챕터에서 다시 다룹니다)
- MERGE 뒤에 SET을 붙인 패턴에 버그가 있어 가격 노드는 "없으면 CREATE, 있으면 SET" 두 단계로 저장합니다

LLM 에이전트 프레임워크 — smolagents에서 pydantic-ai로

에이전트가 도구를 부르는 두 가지 방식

LLM 에이전트는 모델이 "어떤 도구를 어떤 인자로 부를지"를 정하고, 프로그램이 그 도구를 실행해 결과를 다시 모델에 돌려주는 반복 구조입니다. 도구를 부르는 방식은 크게 둘로 나뉩니다.

코드 생성 (smolagents CodeAgent)

- LLM이 **Python 코드**를 쓰고, 그 코드 안에서 도구 함수를 호출
- 변수·for 루프·조건 분기를 쓸 수 있어 유연함
- 대신 **LLM이 만든 코드를 백엔드 프로세스에서 실행**

Tool calling (pydantic-ai)

- LLM이 **JSON 인자**로 도구를 호출
- 인자를 JSON 스키마로 검증, 틀리면 재시도 프롬프트로 되돌림
- 실행되는 코드는 우리가 작성한 도구뿐

smolagents — 처음의 선택

smolagents 는 Hugging Face가 만든 경량 에이전트 프레임워크로, **"최소한의 추상화"** 가 철학입니다. LangChain이 무겁게 느껴져 골랐고, Model + Tool + Agent만 알면 시작할 수 있었습니다. 핵심 기능인 **CodeAgent** 는 도구 호출을 Python 코드로 생성합니다.

pydantic-ai — 지금의 선택

재가동하면서 세 후보를 비교했습니다.

- **smolagents 유지** — 익숙하지만 LLM 생성 코드 실행이라는 구조는 그대로
- **LangChain deepagents** — 계획(todo), 가상 파일시스템, 서브에이전트 같은 장기 작업용 기능이 중심. 도구 10개로 몇 단계면 끝나는 이 챗봇에는 과했습니다
- **pydantic-ai** — 가볍고, OpenAI 호환 엔드포인트를 그대로 쓰며, 도구 인자 스키마 검증이 있고, 코드 실행이 없음 → **선택**

결정적인 이유는 보안이었습니다. smolagents의 로컬 Python 실행기는 샌드박스가 아니었고, 우리 챗봇에 필요한 건 "코드 실행"이 아니라 "읽기 전용 조회 도구를 순서대로 부르는 것"이었습니다.

pydantic-ai의 **run_stream_events** 는 도구 호출과 결과를 이벤트로 흘려 줍니다. 이걸 그대로 프론트엔드의 단계(step) 이벤트로 바꿔서, 기존 SSE 형식을 바꾸지 않고 프레임워크만 교체할 수 있었습니다.

Apache Airflow 기술 소개

Apache Airflow란?

Apache Airflow 는 데이터 파이프라인을 작성하고, 스케줄링하고, 모니터링하는 플랫폼입니다. Airbnb에서 시작해 지금은 Apache 최상위 프로젝트입니다.

- **DAG** — Directed Acyclic Graph. 작업 흐름을 Python 코드로 정의한 것
- **Task** — DAG 안의 개별 작업 단위
- **Operator** — Task가 무엇을 할지 정하는 템플릿 (`PythonOperator` , `ShortCircuitOperator` 등)
- **Scheduler** — 스케줄에 맞춰 DAG를 트리거하는 엔진

Airflow 3.x 구성

지금은 `Airflow 3.3.2` (Python 3.14)를 커스텀 이미지로 씁니다. 2.x 때는 웹서버와 스케줄러를 한 컨테이너에서 돌렸지만, 3.x에서는 역할이 나뉩니다.

- **api-server** — 웹 UI와 API
- **scheduler** — 실행 시점 판단과 Task 실행 (`LocalExecutor`)
- **dag-processor** — DAG 파일 파싱
- **airflow-init** — 메타데이터 DB 마이그레이션과 초기 설정

import 경로도 바뀌어 DAG는 `airflow.sdk` , 표준 오퍼레이터는 `airflow.providers.standard` 에서 가져옵니다.

왜 Airflow인가 — NiFi와의 비교

NiFi

- 실시간 데이터 라우팅에 초점
- GUI 드래그 앤 드롭으로 플로우 구성
- IoT, 로그 수집, 시스템 간 실시간 전달에 강함

Airflow

- 배치 작업 오케스트레이션에 초점
- Python 코드로 워크플로 정의 → Git 버전 관리
- 정기 수집, 배치 ETL에 강함

"거래일 다음 날 아침, KIS Open API에서 전날 확정된 데이터를 가져와 가공해 그래프에 적재한다." 전형적인 배치 작업이고, 수집 로직이 Python 클라이언트이니 Airflow가 자연스러웠습니다.

프로젝트 개요 & 아키텍처

프로젝트 개요

ETF Atlas 는 한국 ETF 정보를 수집·분석하고, AI 챗봇으로 질의하는 웹 서비스입니다.

- **ETF 탐색** — 검색, 테마 태그별 조회, 보유종목, 유사 ETF
- **위치리스트** — 관심 ETF의 보유종목 비중 변화 추적과 알림
- **포트폴리오** — 매수 기록, 목표 비중, 일별 스냅샷 대시보드
- **AI 챗봇** — 자연어로 ETF 데이터 질의
- **관리자** — 챗봇 로그 승인, ETF 태그 수동 지정

전체 아키텍처

Docker Compose로 띄웁니다.

- **db** — PostgreSQL 18 + Apache AGE 1.8.0 + pgvector + pg_trgm
- **backend** — FastAPI (Python 3.14) + pydantic-ai
- **frontend** — React 18 + TypeScript + Vite, nginx가 정적 파일을 서빙하고 `/api` 를 백엔드로 프록시
- **airflow** — 3.3.2, api-server / scheduler / dag-processor / airflow-init

하나의 PostgreSQL에 관계형 테이블, AGE 그래프, pgvector 임베딩이 함께 있습니다. CRUD는 관계형, 관계 탐색은 그래프, 예시 검색은 벡터 — 이게 AGE를 고른 가장 큰 이유입니다.

운영 측면의 정리

- **외부 노출은 프론트엔드 포트(9600) 하나** — 백엔드, DB, Airflow 포트는 모두 `127.0.0.1` 에만 바인딩합니다. 외부 요청은 nginx가 `/api` 로 받아 내부망으로 넘기고, 서버의 Airflow UI는 SSH 터널로 엽니다. Docker의 포트 공개가 방화벽 규칙을 우회할 수 있어서 내린 결정입니다
- **인증은 ID/비밀번호** — Google OAuth를 건너냈습니다. 사용자가 0명이면 첫 화면이 `/setup` 으로 가고, 거기서 만든 계정이 관리자가 됩니다
- **스키마 마이그레이션은 Alembic** — 기존 init SQL을 동결된 기준 리버전(0001)으로 두고, 백엔드가 기동할 때 `alembic upgrade head` 를 실행합니다. 새 DB든 기존 DB든 같은 상태가 됩니다. 0002에서 포트폴리오 외래키에 NOT NULL을 추가했습니다

데이터 수집 — KIS Open API & Airflow

데이터 소스: KIS 하나로

시장 데이터는 한국투자증권 **KIS Open API** 로 통일했습니다. 키 하나로 필요한 대부분을 얻습니다.

- **ETF 목록** — 공개 종목 마스터 파일(`kospi_code.mst`)의 ETF 그룹. 인증이 필요 없습니다
- **ETF/ETN 현재가** — NAV, 순자산, 상장주식수
- **국내주식 기간별 시세** — 일봉, ETF·주식 가격과 영업일 목록(KODEX 200의 일봉 날짜)에 씁니다
- **ETF 구성종목 시세** — 보유종목과 비중
- **국내휴장일조회** — 장 운영 여부, 하루 1회 캐시

KIS가 총보수를 주지 않아 보수율만 네이버 증권에서 가져옵니다. 포트폴리오 현재가 갱신(장중 10분 주기)은 아직 yfinance 를 쓰고 있어, KIS 현재가로 옮기는 것이 남은 과제입니다.

유니버스 — 190개 ETF

- 상장 ETF 약 1,175개
- → 이름 필터: 레버리지·인버스·선물·합성·커버드콜, 채권·금리, 금·은·원자재, 통화, 머니마켓, 리츠, 해외 투자 ETF 제외 → 380여 개
- → 순자산 500억 원 이상 → 190개, 약 156조 원

한 번 유니버스에 들어온 ETF는 빠지지 않습니다. 운용사는 ETF 이름 앞의 브랜드(KODEX, TIGER, ACE 등)로 판별해 `MANAGED_BY` 로 연결합니다.

수집 원칙 세 가지

- **백필하지 않는다** — 첫 실행은 가장 최근 확정 거래일 하루만 수집하고, 이후 매일 씁습니다. 구성종목 API가 날짜 지원을 지원하지 않아 과거 보유종목은 애초에 얻을 수 없습니다
- **확정된 날짜지만 수집한다** — 16시(KST) 전에 돌면 전날까지만 수집합니다. 장중에 일봉을 요청하면 미완성된 당일 봉이 오기 때문입니다
- **구성종목은 비중 상위 30개** — KIS 구성종목 API는 상위 30개만 주고 다음 페이지가 없습니다. KODEX 200은 202종목 중 30개, 비중 합 약 85%입니다. 구성종목은 "XX와 YY가 많이 든 ETF를 수수료순으로" 같은 질의에 쓰는 데이터라 정확한 대사가 목적이 아니어서 그대로 받아들였습니다

Airflow DAG 5개

- **age_sync_universe** (화~토 08:30) — 거래일 목록 → 유니버스·가격 → 구성종목(+ CURRENT_HOLDS 교체) → 주식 가격 → 수익률 → 신규 ETF 규칙 태그 → 수집 기록과 알림
- **rdb_sync_metadata** (평일 08:30) — 종목 마스터 파일 → 관계형 `etfs` 테이블

- **rdb_realtime_prices** (평일 장중 10분마다) — `ShortCircuitOperator` 로 개장 여부를 판단한 뒤 보유 티커 현재가 수집 → 포트폴리오 스냅샷 갱신
- **age_tagging** (토 03:00) — 신규 ETF 테마 태깅 (규칙 + LLM)
- **embed_code_examples** (수동) — 챗봇 few-shot 예시의 질문 일반화와 임베딩

Airflow에서 유용했던 기능

- **Configuration as Code** — DAG가 Python 파일이라 Git으로 관리하고, KIS 클라이언트 같은 수집 코드에 단위 테스트를 붙일 수 있습니다
- **XCom** — "거래일 목록" Task의 결과를 이후 Task들이 받아 씁니다
- **의존성과 병렬 실행** — 유니버스 수집 뒤 구성종목·수익률·태그가 갈라져 병렬로 돕니다
- **자동 재시도** — `retries=3, retry_delay=5분` 설정만으로 간헐적 실패를 흡수
- **Params** — 태깅 DAG의 `full_rebuild` 처럼 수동 실행 시 동작을 바꾸는 인자

그래프 데이터 활용

그래프 모델 (`etf_graph`)

- 노드 — `ETF`, `Stock`, `Price`, `Company`, `Tag`, `User`
- `HOLDS` {date, weight, shares} — 날짜별 보유 이력. 변화 비교에만 씁니다
- `CURRENT_HOLDS` {date, weight, shares} — ETF별 현재 보유종목(최대 30개). 수집할 때마다 교체
- `HAS_PRICE`, `MANAGED_BY`, `TAGGED`, `WATCHES`

"삼성전자를 보유한 ETF 중 반도체 태그가 달린 것" 같은 질의가 MATCH 패턴 하나로 표현됩니다.

왜 `CURRENT_HOLDS`를 따로 두었나

`HOLDS`는 ETF-종목-날짜마다 엣지가 하나씩 생깁니다. 거래일마다 약 4,100개, 1년이면 약 100만 개가 쌓입니다. 예전에는 "현재 보유종목"을 이렇게 구했습니다.

```
MATCH (e:ETF)-[h:HOLDS]→(s:Stock)
WITH e, s, h ORDER BY h.date DESC
WITH e, s, head(collect(h)) AS latest
...
```

ETF마다 이력 전체를 훑는 쿼리라 데이터가 쌓일수록 느려집니다. 지금은 수집 직후 그날 `HOLDS`를 `CURRENT_HOLDS` 로 복사해 두고, 현재 보유종목 조회는 **ETF당 30개 이하의 엣지만** 읽습니다. 조회에 실패한 ETF는 이전 구성종목을 유지하고, 같은 날짜로 다시 돌려도 결과가 같습니다.

유사 ETF

공동 보유종목에서 두 비중 중 작은 값을 합산하고, 기준 ETF의 비중 합으로 나눠 정규화합니다.

```
MATCH (e1:ETF {code: $etf_code})-[h1:CURRENT_HOLDS]→(s:Stock)
      ←[h2:CURRENT_HOLDS]-(e2:ETF)
WHERE e1 <> e2
WITH e2, COUNT(s) AS overlap,
      SUM(CASE WHEN h1.weight < h2.weight
                THEN h1.weight ELSE h2.weight END) AS similarity
WHERE overlap ≥ $min_overlap
RETURN {etf_code: e2.code, name: e2.name,
        overlap: overlap, similarity: similarity}
ORDER BY similarity DESC LIMIT 5
```

구성종목이 상위 30개뿐이라 비중 합이 100%가 아닌 ETF가 많습니다. 정규화하지 않으면 지수형 ETF끼리의 유사도가 낮게 나와서, 자기 비중 합 대비 비율로 나타냅니다.

관계형 DB였다면 자기 조인과 집계가 얽힐 쿼리가, 그래프에서는 **MATCH** 패턴 하나로 읽힙니다.

워치리스트 — 보유종목 변동 감지

관심 ETF의 두 시점 **HOLDS** 를 비교해 **신규 편입, 편출, 비중 증가, 비중 감소** 로 분류합니다. 예를 들어 어떤 ETF가 SK하이닉스 비중을 15%에서 18.5%로 올렸다면 "비중 증가 +3.5%p"로 표시됩니다.

3%p를 넘는 변화가 있으면 알립니다. 수집이 끝나면 PostgreSQL **LISTEN/NOTIFY** → 백엔드 SSE로 앱 상단에 알림 점이 뜨고, 설정되어 있으면 관리자의 **WATCHES** 를 따라 Discord로도 보냅니다.

ETF 테마 태깅

태그 27개, 세 단계

반도체, AI, 2차전지, 방산, 고배당 같은 테마 태그 27개를 씁니다. 태그는 순서대로 붙입니다.

- **지수 규칙** — "200", "코스피", "KRX300", "150", "코스닥" 같은 이름 패턴으로 **코스피** / **코스닥** . 이 두 태그는 규칙으로만 붙습니다
- **키워드 규칙** — 이름에 "반도체", "배터리", "원자력" 등이 있으면 해당 태그
- **LLM 분류** — 나머지는 ETF 이름과 보유종목 상위 10개를 근거로 LLM이 structured output으로 태그를 고릅니다. 5개씩 묶어 보내고, 추론 중 토큰이 바닥난 배치는 thinking을 끄고 한 번 더 시도합니다

시장 전체를 담는 ETF처럼 맞는 테마가 없으면 **태그를 붙이지 않아도 되게** 했습니다. 가장 가까운 태그를 억지로 고르게 하면 오분류가 늘었습니다.

신규 ETF만 태깅

예전에는 매주 전체 ETF를 다시 태깅했는데, LLM 결과가 실행마다 달라 같은 ETF의 태그가 주마다 바뀌었습니다(예: "전력·원전"이었다가 "전력·건설"). 지금은 **tagged_at** 이 없는 ETF, 즉 새로 편입된 ETF만 태깅합니다. 규칙이나 태그 목록을 바꿨을 때는 **full_rebuild** 파라미터로 수동 실행해 전체를 다시 태깅합니다.

관리자 수동 지정

관리자 페이지의 **"ETF 태그"** 탭에 모든 ETF와 태그가 나옵니다. 셀렉트 박스로 추가하고 배지에서 제거하면 그 목록이 **manual_tags** 로 고정되고, 태깅 DAG는 이 ETF에 규칙과 LLM을 적용하지 않고 지정값을 그대로 씁니다.

우량주 는 수동으로만 붙는 태그입니다. "산업 테마 없이 시가총액 상위·성장성·ESG 등 선별 기준으로 대형주를 골라 담는 ETF"를 뜻하고, 지수를 폭넓게 따라가는 ETF는 계속 **코스피** 로 둡니다. 기준이 사람의 판단에 가까워 LLM에 맡기지 않았습니다.

AI 챗봇

전체 구조

`ChatService` 는 pydantic-ai **tool-calling 에이전트** 하나입니다. LLM은 `qwen38-27b` , 임베딩은 `embedding-gemma-300m` (768차원)이고 둘 다 LiteLLM 프록시(OpenAI 호환)를 거칩니다. 도구는 10개입니다.

- 검색: `etf_search` , `stock_search` , `list_tags`
- 정보: `get_etf_info` , `get_etf_prices` , `get_stock_prices` , `get_holdings_changes`
- 분석: `find_similar_etfs` , `compare_etfs` , `graph_query` (Cypher 직접 실행)

모든 도구가 Apache AGE 그래프를 조회합니다. 그래프와 에이전트가 여기서 만납니다.

동작 흐름

Q: 삼성전자를 가장 많이 보유한 ETF는?

[Step 1] `stock_search(query="삼성전자")`

[Step 2] `graph_query(cypher="MATCH (e:ETF)-[h:CURRENT_HOLDS]`

`→(s:Stock {code: '005930'})`

`RETURN {etf_code: e.code, etf_name: e.name, weight: h.weight}`

`ORDER BY h.weight DESC LIMIT 5")`

[답변] 마크다운 표

모델이 도구 호출을 정하면 프로그램이 실행하고 결과를 돌려주는 과정을, 모델이 도구 없이 답할 때까지 반복합니다. 질문 하 나당 모델 요청 수에 상한(15회)을 두었습니다. `run_stream_events` 로 받은 도구 호출과 결과를 단계 이벤트로 바꿔 화면 의 접이식 패널에 실시간으로 보여 줍니다.

사용자는 AI가 어떤 도구를 어떤 인자로 불러 답을 만들었는지 **투명하게** 볼 수 있습니다.

Tool 정의

도구마다 `name` , `description` , `inputs` (JSON 스키마), `forward()` 를 정의하면 pydantic-ai 도구로 변환됩니다.

```
class ETFSearchTool(ChatTool):
    name = "etf_search"
    description = "ETF를 이름이나 코드로 검색합니다. ... "
    inputs = {
        "query": {"type": "string", "description": "검색 키워드"},
        "limit": {"type": "integer", "nullable": True,
                  "description": "최대 결과 수 (기본값 10)"},
    }

    def forward(self, query: str, limit: int = 10) → str: ...

# ChatTool.as_agent_tool() → pydantic_ai Tool.from_schema( ... )
```

핵심은 description입니다. 모델은 이 설명을 보고 도구를 고르고 인자를 채웁니다. 인자가 스키마에 맞지 않으면 재 시도 프롬프트로 모델에 되돌아가고, 화면에는 오류 단계로 표시됩니다.

graph_query의 안전장치

LLM이 쓴 Cypher를 그대로 실행하는 도구라 가장 위험합니다.

- 별도 연결에서 `SET TRANSACTION READ ONLY` 트랜잭션으로 실행, 끝나면 롤백
- `statement_timeout` 10초
- `$$` 와 `;` 가 든 쿼리는 거부 (SQL 따옴표 탈출 차단)
- 쿼리 오류 메시지를 LLM에 돌려줘 스스로 고치게 함

또 하나, 도구 결과는 사람이 읽기 좋게 %와 억원 단위로 서식을 붙여 돌려주는데, 이걸 본 모델이 저장된 값도 문자열이라고 여기고 조건을 문자열로 쓴 적이 있습니다. 그래서 `graph_query` 설명에 "값은 숫자로 저장돼 있고 서식은 표시용"이라고 명시했습니다.

Few-shot — Cypher 패턴 예시

시드 예시 `38개` 를 도구 호출 순서로 적어 둡니다. 핵심은 Cypher 구조이고, 고유명사는 자리표시자(`<종목명1>` , `<ETF명>` , `<운용사>`)로 둡니다. 종목은 Cypher 안에서 이름으로 바로 찾습니다. 원본은 `docker/db/seed/code_examples.py` 이고 여기서 SQL을 생성합니다.

```
Q: 삼성전자와 SK하이닉스가 많이 들어있는 ETF를 수수료순으로 정렬해줘
(질문은 일반화해서 임베딩, 코드는 자리표시자)

graph_query(cypher="MATCH (e:ETF)-[h:CURRENT_HOLDS]→(s:Stock)
WHERE s.name IN ['<종목명1>', '<종목명2>']
WITH e, count(s) AS matched, sum(h.weight) AS weight_sum
WHERE matched = 2
RETURN {code: e.code, name: e.name, weight_sum: weight_sum,
        expense_ratio: e.expense_ratio}
ORDER BY e.expense_ratio ASC, weight_sum DESC LIMIT 10")
```

예시의 질문은 일반화해서("삼성전자를..." → "특정 종목을...") 임베딩해 둡니다. 질문이 들어오면 같은 방식으로 일반화한 뒤(thinking을 끄면 0.2~0.4초, 켜를 때와 결과가 같았습니다) pgvector로 가까운 예시 3개를 찾아 일반화된 질문과 함께 프롬프트에 넣습니다.

- "현대차와 기아..." — 엉뚱한 예시(거리 0.29) 대신 맞는 예시가 거리 0.19로 매칭
- "에코프로비엠과 포스코퓨처엠..." — 매칭 없음 → 거리 0.16

관리자가 승인한 채팅 로그도 같은 도구 호출 형식의 예시로 추가됩니다.

설계 변천사

1단계: ReAct만 — CodeAgent의 ReAct만 썼더니 **Cypher 정확도가 낮았습니다**. AGE의 Cypher는 Neo4j와 미묘하게 다른데, 모델이 그걸 알 리 없었습니다.

2단계: Cypher Few-shot — Cypher 예시 100개를 pgvector로 찾아 넣으니 단순 질의는 좋아졌지만, 도구를 7~8번 불러야 하는 복잡한 질문에서는 방향을 잃었습니다.

3단계: 분류기 + Plan-Execute-Summarize — 예상 호출 수가 많으면 계획을 JSON으로 세우고 순서대로 실행한 뒤 요약하게 했습니다. 안정적이었지만 구조가 무거웠습니다.

4단계: ReAct + Python 코드 예시 — "CodeAgent는 for 루프를 쓸 수 있는데 외부 플래너가 왜 필요한가?" PES를 건너내고, 예시를 Python 코드로 바꿨습니다.

5단계(현재): tool-calling + 일반화된 Cypher 패턴 — 프레임워크를 pydantic-ai로 바꾸고, 예시는 다시 Cypher 패턴으로 돌아왔습니다. 대신 자리표시자로 일반화하고, 검색도 일반화된 질문끼리 비교합니다.

왜 진자가 다시 Cypher 쪽으로 돌아왔나. 예시가 실제로 도움을 준 부분은 Python 문법이 아니라 그 안에 든 Cypher 구조였습니다. 복잡한 조합은 Cypher 한 번이나 여러 도구 호출로 충분히 풀렸고, 그렇다면 읽기 전용 도구 호출로 같은 일을 할 수 있는데 LLM이 만든 Python을 실행하는 것은 불필요한 위험이었습니다. 고유명사가 박힌 예시가 엉뚱하게 매칭되는 문제는 자리표시자와 질문 일반화로 풀었습니다.

포트폴리오 관리 & 대시보드

포트폴리오 관리

사용자가 ETF와 주식의 매수 기록을 직접 입력하면 현재가로 평가액을 계산합니다. 목표 비중을 정하면 실제 비중과의 차이와 리밸런싱 계산을 보여 줍니다. 보유 수량·평균 단가와 스냅샷 금액은 앱 단에서 `AES-256-GCM` 으로 암호화해 저장합니다.

대시보드

`portfolio_snapshots` 테이블에 포트폴리오별 일별 스냅샷이 쌓이고, 대시보드는 `Recharts` 로 가치 변화를 그립니다. 스냅샷은 장중 10분마다 도는 실시간 가격 DAG가 가격 수집 직후 갱신하며, 포트폴리오마다 스냅샷 기록을 켜고 끌 수 있습니다. 같은 날은 `UNIQUE(portfolio_id, date)` 로 덮어씁니다.

과거 이력을 백필하지 않으므로 추이 차트는 데이터가 쌓인 만큼만 보입니다.

이 기능은 관계형 테이블을 씁니다. CRUD와 트랜잭션 중심 데이터는 **관계형**으로, 관계 탐색은 **그래프**로 나눈 설계입니다.

시행착오

"오늘"은 언제인가 — 기준일과 역등성

"장 마감 후에 그날 데이터를 저장한다." 단순히 보이지만 기준일과 수집일을 구분하지 않고 출발해서 여러 번 고쳐야 했습니다.

- **데이터 없음 ≠ 값이 0** — 휴장일 다음 실행에서 새 가격이 없자 수익률이 0%로 덮어써졌습니다. 새 데이터가 없으면 기존 값을 건드리지 않게 고쳤습니다
- **요청한 날짜 ≠ 데이터의 날짜** — 비거래일을 요청하면 직전 거래일 데이터가 오는데, 요청 날짜로 저장했다가 가격 노드가 중복됐습니다. 지금은 영업일 목록 자체를 KIS 일봉이 돌려주는 실제 날짜로 만들고, 그 날짜를 XCom으로 하위 Task에 넘깁니다
- **수동 재실행 시 중복 알림** — DB 쓰기는 `ON CONFLICT DO NOTHING` 으로 역등했지만 알림은 매번 나갔습니다. `rowcount` 로 새 행이 생겼을 때만 알립니다
- **스냅샷의 기준** — 스냅샷 전용 DAG가 가격 수집보다 먼저 돌아 어제 가격으로 오늘 스냅샷이 찍히기도 했습니다. 가격 수집 바로 뒤에 스냅샷을 갱신하는 한 DAG로 합치고 날짜별 UPSERT로 정리했습니다

장중의 미완성 일봉

KIS로 옮긴 뒤 오전 10시에 수집 DAG를 수동으로 돌렸더니, KIS 일봉이 장중에도 당일 봉을 돌려줘서 **장중 가격이 그날 종가로 저장** 됐습니다. 다음 실행은 그 날짜를 이미 수집한 것으로 보고 넘어가므로 저절로 고쳐지지도 않는 구조였습니다. 이전 코드는 수집 범위를 항상 "오늘"까지 잡아서, 아침 정기 실행도 같은 위험을 안고 있었습니다. 수집 범위의 끝을 "종가가 확정된 마지막 날"로 잘라, 16시 전이면 전날까지만 수집하게 했습니다.

이력이 쌓이는 관계 — HOLDS에서 CURRENT_HOLDS로

날짜별 이력 엮지는 하루 약 4,100개씩 늘어납니다. "최신 보유종목" 쿼리가 ETF마다 이력 전체를 훑고 있었고, 지금은 괜찮아도 1년 뒤면 약 100만 개를 훑게 됩니다. 현재 상태를 담은 `CURRENT_HOLDS` 를 따로 두고 이력은 비교에만 쓰도록 나눴습니다. **"현재"와 "이력"은 같은 관계라도 저장을 분리** 하는 편이 낫습니다.

LLM 태깅의 비결정성

같은 ETF가 주마다 다른 태그를 받았습니다. LLM 출력은 실행마다 달라질 수 있다는 걸 전제로 설계를 바꿨습니다. **한 번 붙인 태그는 고정** 하고 신규 ETF만 태깅하며, 사람이 지정한 태그(`manual_tags`)는 자동 태깅이 건드리지 않습니다. 전체 재태깅은 명시적으로 요청할 때만 합니다.

이름 기반 휴리스틱의 구멍

- **은 ETF가 유니버스에 들어옴** — 제외 키워드가 "은현물"뿐이라 "1Q 은액티브"가 통과했습니다. 그렇다고 "은"을 키워드로 넣으면 "은행" ETF까지 빠집니다. 단어 첫머리의 "은"(뒤에 "행"이 오지 않는)만 잡는 정규식으로 고쳤습니다

- **운용사 "기타" 7개** — IBK, FOCUS, DAISHIN 같은 브랜드가 매핑 표에 없어 7개 ETF가 "기타"로 연결됐습니다. 마스터 파일에 남은 미매핑 브랜드까지 함께 추가했습니다

이름은 편리한 1차 필터지만 규칙에 없는 이름은 조용히 새어 나갑니다. "기타"나 미분류 개수처럼 **새는 것을 드러내는 지표**를 주기적으로 확인해야 합니다.

LLM이 쓴 쿼리의 보안

처음 graph_query는 postgres 슈퍼유저 권한으로, CREATE·DELETE 같은 **키워드 블랙리스트**에만 기대고 있었습니다. 점검해 보니 TRUNCATE·ALTER는 목록에 없었고, 쿼리에 **\$\$**를 넣으면 Cypher를 감싼 SQL 따옴표를 빠져나가 임의의 SQL을 실행할 수 있었습니다. 지금은 **읽기 전용 트랜잭션 + 실행 시간 제한 + \$\$ · ; 거부**로 바꿨습니다. 금지어를 나열하는 대신 DB가 쓰기를 거부하게 만드는 쪽이 우회할 틈이 적습니다.

그래프 vs 관계형 — 어디에 뭘 넣을까

그래프에 적합

- 관계 탐색 (ETF-종목 보유, 유사 ETF, 태그)
- 시계열 + 관계 (날짜별 보유 비중 변화)
- 다대다 관계 (사용자-ETF 워치리스트)

관계형에 적합

- 단순 CRUD (사용자, 포트폴리오, 매수 기록)
- 집계·정렬 중심 조회 (스냅샷, 평가액)
- 트랜잭션이 중요한 데이터 (인증)

대표 사례가 가격 데이터입니다. 처음에는 `etf_prices`, `stock_prices` 관계형 테이블에 뒀지만, 챗봇 도구가 모두 그래프를 조회하다 보니 SQL과 Cypher를 오가야 했습니다. `Price` 노드와 `HAS_PRICE` 관계로 옮기고 테이블을 없앴습니다. **저장소는 "주로 어디서 조회하느냐"에 맞춰야** 한다는 교훈입니다.

"전부 그래프"나 "전부 관계형"이 아니라 데이터 특성에 맞게 섞어 쓰는 것이 현실적입니다. 그 분리를 한 DB 안에서 할 수 있다는 것이 AGE의 장점입니다.

회고 & 마무리

기술 선택 회고

이 프로젝트에서 직접 겪은 것을 바탕으로 한 팀의 평가입니다.

Apache AGE

Good

PostgreSQL 하나로 **관계형 + 그래프 + 벡터 검색** 을 운영합니다. Cypher도 결국 SQL 안에서 돌기 때문에 읽기 전용 트랜잭션, statement_timeout, Alembic 같은 PostgreSQL 도구를 그대로 쓸 수 있었습니다.

Bad

Cypher 지원이 불완전합니다(WITH 별칭 정렬, MERGE+SET). \$\$ 따옴표 안에 쿼리를 넣는 구조라 외부 입력을 다룰 때 신경 쓸 게 많습니다. 이력이 쌓이는 관계는 모델링을 따로 고민해야 했고, 커뮤니티와 자료가 적습니다.

Recommend

이미 PostgreSQL을 쓰고 있고 그래프 쿼리가 전체 워크로드의 일부일 때.

pydantic-ai (tool-calling)

Good

가볍고 OpenAI 호환 엔드포인트에 바로 붙습니다. 도구 인자를 스키마로 검증하고 틀리면 모델에 되돌려 주며, 이벤트 스트림 덕에 기존 SSE 형식을 유지한 채 프레임워크를 바꿀 수 있었습니다. LLM이 만든 코드를 실행하지 않아도 됩니다.

Bad

CodeAgent처럼 루프·변수로 결과를 가공할 수 없어 정렬·필터는 모델이나 Cypher에 맡겨야 합니다. 도구들이 DB 세션 하나를 공유해 실행은 순차로 합니다. 추론 모델은 토큰 예산을 넉넉히 줘야 합니다.

Recommend

읽기 위주 도구 10개 안팎, 몇 단계면 끝나는 질의형 에이전트. 계획·파일시스템·서브에이전트가 필요한 장기 작업이라면 더 무거운 프레임워크를 검토할 만합니다.

Apache Airflow 3.x

Good

작업 흐름을 Python 코드로 관리하고 테스트할 수 있습니다. 웹 UI, 재시도, 의존성 기반 병렬 실행, XCom, Params가 설정만으로 동작합니다.

Bad

컨테이너가 api-server, scheduler, dag-processor, init으로 늘었고 2.x에서 올릴 때 import 경로와 이미지 구성을 손봐야 했습니다. "Hello World"까지 단계가 많습니다.

Recommend

Python 기반 배치 ETL과 정기 수집. 실시간 데이터 라우팅이 핵심이라면 NiFi 쪽이 맞습니다.

KIS Open API (데이터 소스)

Good

키 하나로 ETF 목록(마스터 파일은 키도 불필요), 현재가, 일봉, 구성종목, 휴장일까지 얻습니다. 주식 일봉을 종목당 기간 단위로 한 번에 받아 호출 수가 줄었습니다.

Bad

구성종목은 상위 30개만, 날짜 지정 불가라 과거 백필이 안 됩니다. 장중에는 미완성 일봉이 옵니다. 토큰 발급·초당 호출 제한을 다뤄야 하고, 순자산 단위처럼 문서에 없는 부분이 있으며, 총보수는 제공하지 않습니다.

Recommend

국내 시세를 일 단위로 꾸준히 쌓는 개인·리서치 프로젝트. 정확한 전체 구성종목이나 과거 이력이 필요하다면 다른 소스를 함께 검토해야 합니다.

감사합니다.

질문 받겠습니다.